

August 25, 2024

1

1.1 Case Study Introduction

This case study focuses on one of the leading Indian private sector banks' endeavors to enhance personal loan campaigns through Data Exploration and Predictive Modeling.

1.2 Market Context

In the dynamic landscape of the Indian banking sector, competition is fierce, and customer preferences constantly evolve. The personal loan market in India is a \$20+ billion industry with interest rates ranging from 10% to 20%. This market primarily attracts younger working professionals who often apply online, while lenders face competition, credit scoring challenges, and regulatory oversight.

With a network of 500+ branches, our client operates in a highly competitive market where customer acquisition, retention, and cross-selling are critical for sustained growth. India's diverse economic and cultural landscape necessitates a flexible and data-driven marketing approach. Understanding and catering to the specific financial needs of a large and diverse customer base is essential for our client's continued success.

With a majority of customers being depositors, the bank's market presence is solidified as a trusted repository for savings. Yet, the bank faces a challenge in diversifying its services and engaging these depositors in lucrative products like personal loans, which require a different marketing strategy. The bank's market context includes a quest for innovative marketing techniques to target potential customers effectively.

1.3 Bank Background

██████████ a leading Indian bank, is a prominent player in India's financial sector. With a reputation for safety and reliability, the bank has established a significant foothold in the Indian banking industry.

- **Branches:** Operates with a strong presence, boasting over 300 branches nationwide.
- **Customer Base:** The bank's customer base surpasses 11.5 million individuals, primarily characterized as depositors.
- **Services:** The bank's proficiency in accumulating and preserving deposits serves as a financial custodian to millions.

The bank sees this as an opportunity to capitalize on the trust and diversify its portfolio by offering personal loans to depositors. The bank's rich history, extensive reach, and solid foundation provide

a solid backdrop for leveraging data-driven marketing strategies to explore new avenues and enhance customer engagement.

1.4 Business Challenge

The primary challenge faced by our client is to identify potential customers within its vast customer base who are likely to accept personal loan offers. This challenge involves understanding the factors that influence a customer's decision to take a personal loan and developing a data-driven strategy to target these potential customers effectively. The bank aims to increase the success rate of its marketing campaigns by refining its approach.

1.5 Starting the Project!

1.6 Importing required libraries

```
[1]: import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.ensemble import VotingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
import seaborn as sns
```

1.7 Loading the Dataset

```
[2]: #Loading the Dataset
bank_data = pd.read_csv('loan_dataset.csv')
```

1.8 Exploratory Data Analysis

Understanding the dataset and its values

```
[3]: bank_data.head()
```

```
[3]:
```

	Unnamed: 0	ID	Pin-code	age	Fam members	Education	\
0	0	10001	110001	23	4.0	Under Graduate	
1	1	10002	110014	43	3.0	Under Graduate	
2	2	10003	110014	37	1.0	Under Graduate	
3	3	10004	110003	33	1.0	Graduate	
4	4	10005	110004	33	4.0	Graduate	

	T.Experience	Income	Mortgage	Fixed Deposit	Demat	Net Banking	Loan
0	1.0	392000	0	no	yes	no	no

1	19.0	272000	0	no	yes	no	no
2	15.0	88000	0	no	no	no	no
3	9.0	800000	0	no	no	no	no
4	8.0	360000	0	no	no	no	no

It seems that the dataset had one serial number column, without any column name, at the starting position. It is unnecessary to us and will be removed.

```
[4]: #Removing the first column
df = bank_data.drop(columns='Unnamed: 0')
```

From the first five rows of the loan dataset, we can understand that there are 11 columns and in these 11 columns There are 5 continuous columns. -> ID -> Pin-code -> age -> Fam members -> T.Experience -> Income There are 5 binary classification columns, namely -> Mortgage -> Fixed Deposit -> Demat -> Net Banking -> Loan Finally, there is one categorical column of Education

Now let's perform deeper checking of the data and even Data Cleaning if required
Checking for missing values

```
[5]: print(df.isnull().sum())
```

```
ID                0
Pin-code          0
age               0
Fam members       0
Education         0
T.Experience      0
Income            0
Mortgage          0
Fixed Deposit     0
Demat             0
Net Banking       0
Loan              0
dtype: int64
```

From this, we can see that, there is no missing data. Dataset is very well made.

1.8.1 We will now start transforming our data into what the model can understand and learn from

The first hurdle we will tackle, are the binary categorical columns

```
[6]: bin_ca = df[['Fixed Deposit', 'Demat', 'Net Banking']]
bin_ca.head()
```

```
[6]:   Fixed Deposit  Demat  Net Banking
0             no    yes           no
1             no    yes           no
2             no    no            no
```

3	no	no	no
4	no	no	no

Fixed Deposit Demat Net Banking

```
[7]: # Using label encoding for binary categorical columns
le = LabelEncoder()
```

```
[8]: #Transforming the columns
df['Fixed Deposit'] = le.fit_transform(df['Fixed Deposit'])
df['Demat'] = le.fit_transform(df['Demat'])
df['Net Banking'] = le.fit_transform(df['Net Banking'])
```

```
[9]: bin_ca = df[['Fixed Deposit', 'Demat', 'Net Banking']]
bin_ca.head()
```

```
[9]:
```

	Fixed Deposit	Demat	Net Banking
0	0	1	0
1	0	1	0
2	0	0	0
3	0	0	0
4	0	0	0

Second hurdle is the Education column To convert this categorical column into something our model can understand, we need to create as many new columns as the types of elements in the column

```
[10]: df['Education'].value_counts()
```

```
[10]: Education
Under Graduate    2096
Post Graduate     1501
Graduate          1403
Name: count, dtype: int64
```

The categorical column, Education, contains Three different types of answers, namely 1. Under Graduate 2. Post Graduate 3. Graduate here, we can see that there is one value known as Graduate but, from what we know about the general education system, there is no such thing as Graduate. Students are either UG or PG. Therefore, Graduate is either a mistype or it means the people who hold a doctoral degree(Ph.D. equivalent). It might also mean people who have done some sort of graduation or are degree holders but not Under Graduate level. but, it can also mean people who hold a degree higher than post graduate but not doctorate level since the number of Graduate are less than both UG & PG. For example, (law degrees) J.D. or (medical degrees) M.D.

However, due to lack of information in this case, I am going to leave it as it is.

Three new columns will be created with values 0 & 1.

```
[11]: # One-hot encoding for Education
df = pd.get_dummies(df, columns=['Education'])
```

1.8.2 After the preliminary data transformation, this is how our new top five rows look

```
[12]: df.head()
```

```
[12]:      ID  Pin-code  age  Fam members  T.Experience  Income  Mortgage  \
0  10001    110001   23         4.0          1.0   392000         0
1  10002    110014   43         3.0         19.0   272000         0
2  10003    110014   37         1.0         15.0    88000         0
3  10004    110003   33         1.0          9.0   800000         0
4  10005    110004   33         4.0          8.0   360000         0
```

```
      Fixed Deposit  Demat  Net Banking Loan  Education_Graduate  \
0                0      1              0  no                False
1                0      1              0  no                False
2                0      0              0  no                False
3                0      0              0  no                 True
4                0      0              0  no                 True
```

```
      Education_Post Graduate  Education_Under Graduate
0                False                True
1                False                True
2                False                True
3                False                False
4                False                False
```

Label Encoding the new education columns

```
[13]: df['Education_Graduate'] = le.fit_transform(df['Education_Graduate'])
df['Education_Post Graduate'] = le.fit_transform(df['Education_Post Graduate'])
df['Education_Under Graduate'] = le.fit_transform(df['Education_Under_
↳ Graduate'])
```

```
[14]: df['Loan'] = le.fit_transform(df['Loan'])
```

```
[15]: df.head()
```

```
[15]:      ID  Pin-code  age  Fam members  T.Experience  Income  Mortgage  \
0  10001    110001   23         4.0          1.0   392000         0
1  10002    110014   43         3.0         19.0   272000         0
2  10003    110014   37         1.0         15.0    88000         0
3  10004    110003   33         1.0          9.0   800000         0
4  10005    110004   33         4.0          8.0   360000         0
```

```
      Fixed Deposit  Demat  Net Banking Loan  Education_Graduate  \
```

0	0	1	0	0	0
1	0	1	0	0	0
2	0	0	0	0	0
3	0	0	0	0	1
4	0	0	0	0	1

	Education_Post Graduate	Education_Under Graduate
0	0	1
1	0	1
2	0	1
3	0	0
4	0	0

```
[16]: df_ = df[['age', 'Fam members', 'Income', 'Mortgage']]

min_values = df_.min()
max_values = df_.max()

min_max_df = pd.DataFrame({'Min': min_values, 'Max': max_values})
print(min_max_df)
```

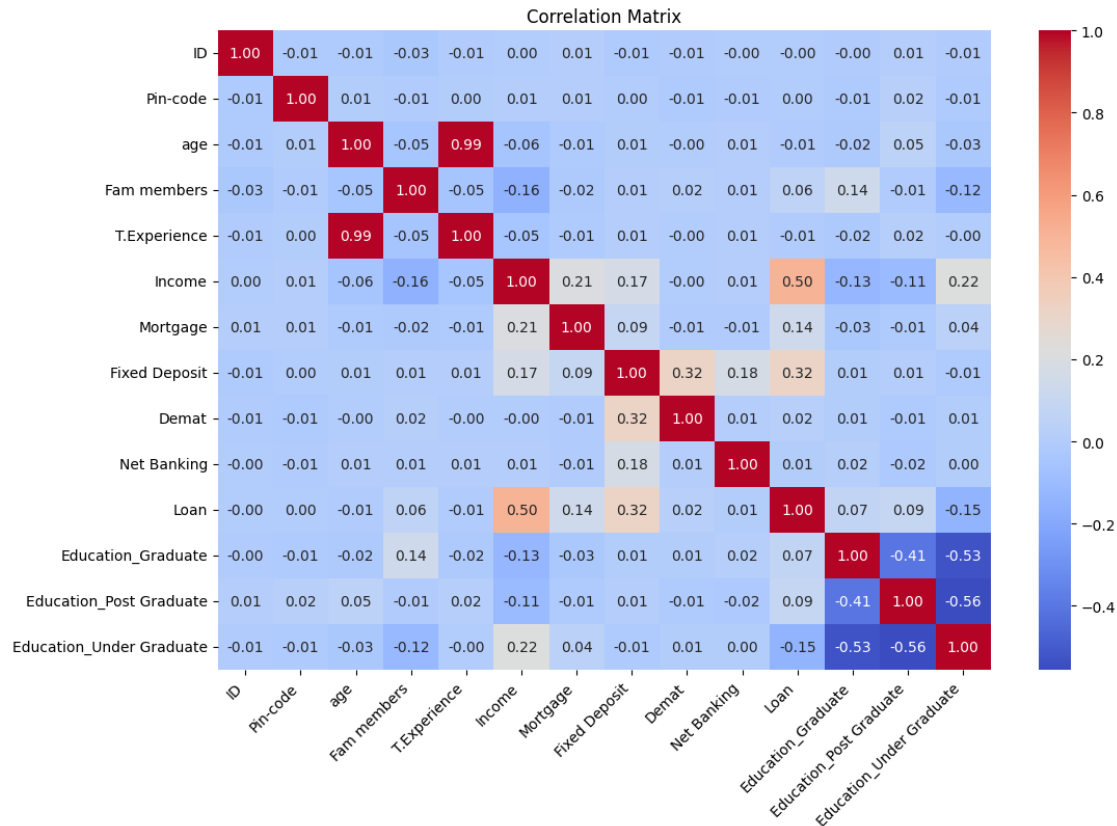
	Min	Max
age	21.0	65.0
Fam members	1.0	4.0
Income	64000.0	1792000.0
Mortgage	0.0	5080000.0

Bank Customer Profile Analysis:

1.9 Visualization

After the preliminary data cleaning and transformation, we will now try to understand the data deeper and try to ascertain the relationships each variable has.

```
[17]: correlation_matrix = df.corr()
plt.figure(figsize=(12, 8)) # Increase figure size for readability
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm',
            annot_kws={"size": 10})
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(fontsize=10)
plt.title('Correlation Matrix')
plt.show()
# Finding the correlation with the target variable
correlation_with_target = df.corr()['Loan'].sort_values(ascending=False)
print(correlation_with_target)
```



```

Loan                1.000000
Income              0.502462
Fixed Deposit       0.316355
Mortgage            0.142095
Education_Post Graduate 0.090212
Education_Graduate  0.071491
Fam members        0.061367
Demat               0.021954
Net Banking         0.006278
Pin-code            0.002344
ID                  -0.000576
T.Experience        -0.007413
age                 -0.007726
Education_Under Graduate -0.148894
Name: Loan, dtype: float64

```

We can infer a few things from this correlation heat map Individuals with higher incomes are more likely to have larger mortgages, own properties, and invest in financial instruments like fixed deposits and demat accounts.

Education level plays a role in determining income, with higher education often leading to higher

earnings.

Key Observations Regarding Loan:

Loan and Income There's a moderate positive correlation between Loan and Income. This suggests that individuals with higher incomes tend to take out larger loans.

Loan and Education There's a weak positive correlation between Loan and Education_Graduate. This might indicate that individuals with graduate degrees are slightly more likely to take out loans, possibly for higher education or business ventures.

Loan and Fixed Deposit There's a weak negative correlation between Loan and Fixed Deposit. This could suggest that individuals with larger fixed deposits might be less likely to take out loans, as they may have alternative sources of funds.

Loan and Net Banking There's a moderate positive correlation between Loan and Net Banking. This might imply that individuals who are more comfortable with online banking are more likely to apply for loans online, which could lead to higher loan amounts.

Interpreting the Correlations:

Based on these correlations, we might infer that:

Individuals with higher incomes are more likely to take out larger loans, possibly for home purchases, investments, or business ventures. Online banking might facilitate the loan application process, leading to higher loan amounts. Individuals with significant savings might be less likely to need loans.

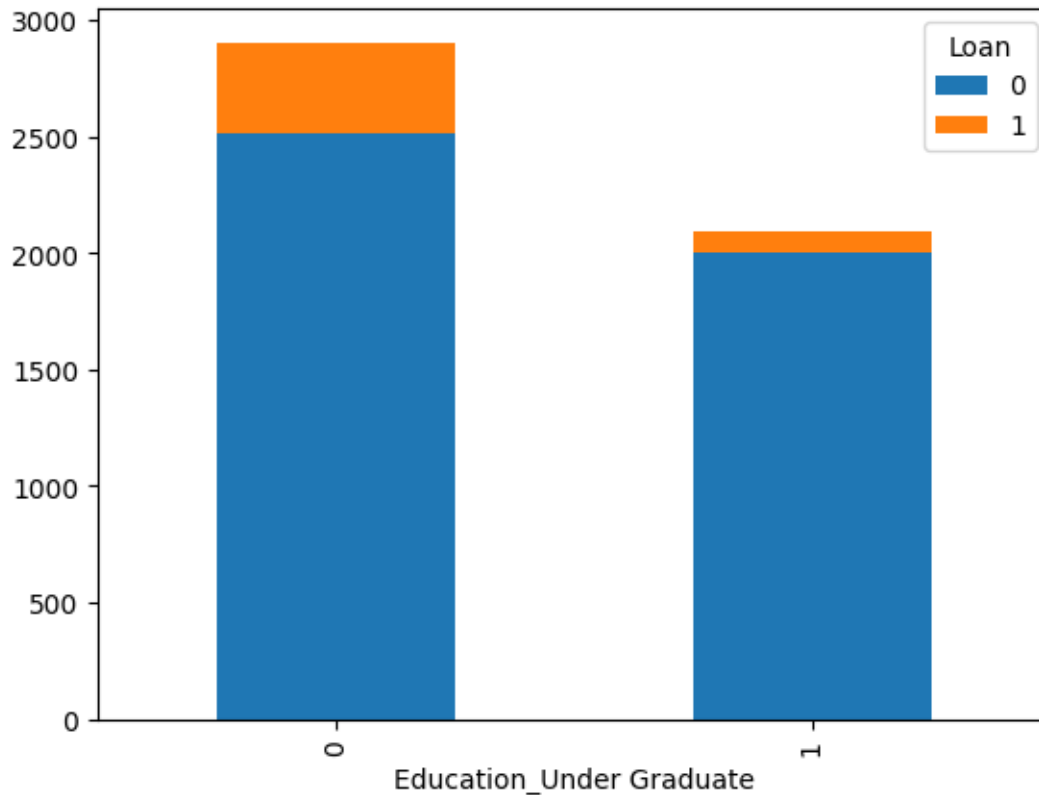
Positive Correlations: The positive correlations between Loan and Income & Loan and Net Banking suggest that these factors might influence the amount or likelihood of taking out a loan.

Negative Correlation: The weak negative correlation between Loan and Fixed Deposit indicates that individuals with substantial savings might be less reliant on loans.

However correlation does not also mean causation While correlations can suggest relationships between variables, they don't prove causation. Further analysis is needed in this case.

1.9.1 Analyzing how much part does Education plays in receiving loans

```
[18]: pd.crosstab(df['Education_Graduate'], df['Loan']).plot(kind='bar', stacked=True)
plt.show()
pd.crosstab(df['Education_Post Graduate'], df['Loan']).plot(kind='bar',
    ↪stacked=True)
plt.show()
pd.crosstab(df['Education_Under Graduate'], df['Loan']).plot(kind='bar',
    ↪stacked=True)
plt.show()
```



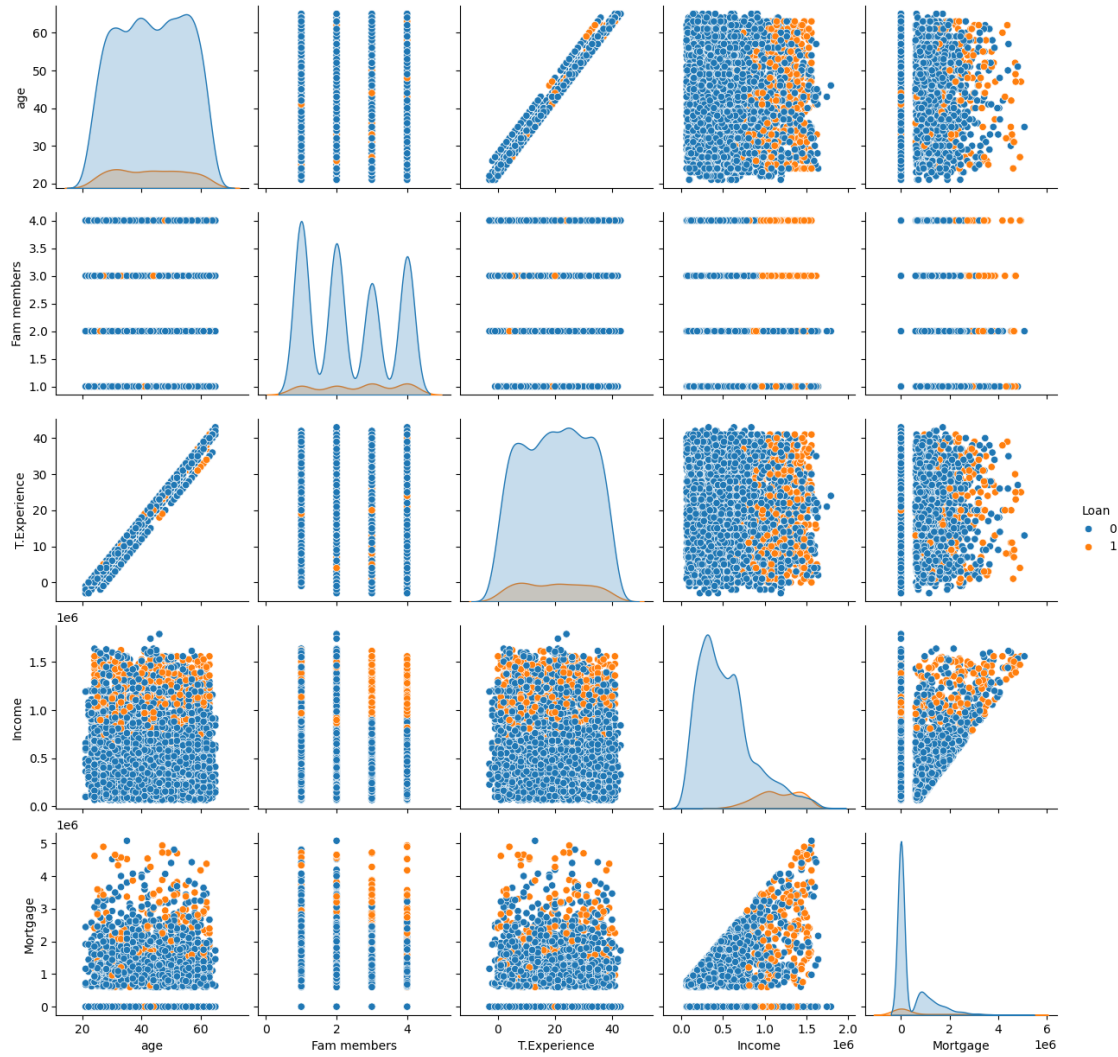
Key Findings: Undergraduates: A higher proportion of individuals who are not undergraduates (category 0) have taken out loans compared to those who are undergraduates (category 1).

Post-Graduates: Similar to undergraduates, a higher proportion of individuals who are not post-graduates (category 0) have taken out loans compared to those who are post-graduates (category 1).

Graduates: Again, a higher proportion of individuals who are not graduates (category 0) have taken out loans compared to those who are graduates (category 1).

1.9.2 Detailed Analysis

```
[19]: sns.pairplot(df[['age', 'Fam members', 'T.Experience', 'Income', 'Mortgage', 'Loan'], hue='Loan')
plt.show()
```



Key Observations: Distributions: The diagonal plots show the distribution of each variable.

Age: Appears to be normally distributed with a peak around 30.

Fam members: Skewed to the right, indicating a few individuals have larger families.

T.Experience: Skewed to the right, suggesting some individuals have significantly more experience.

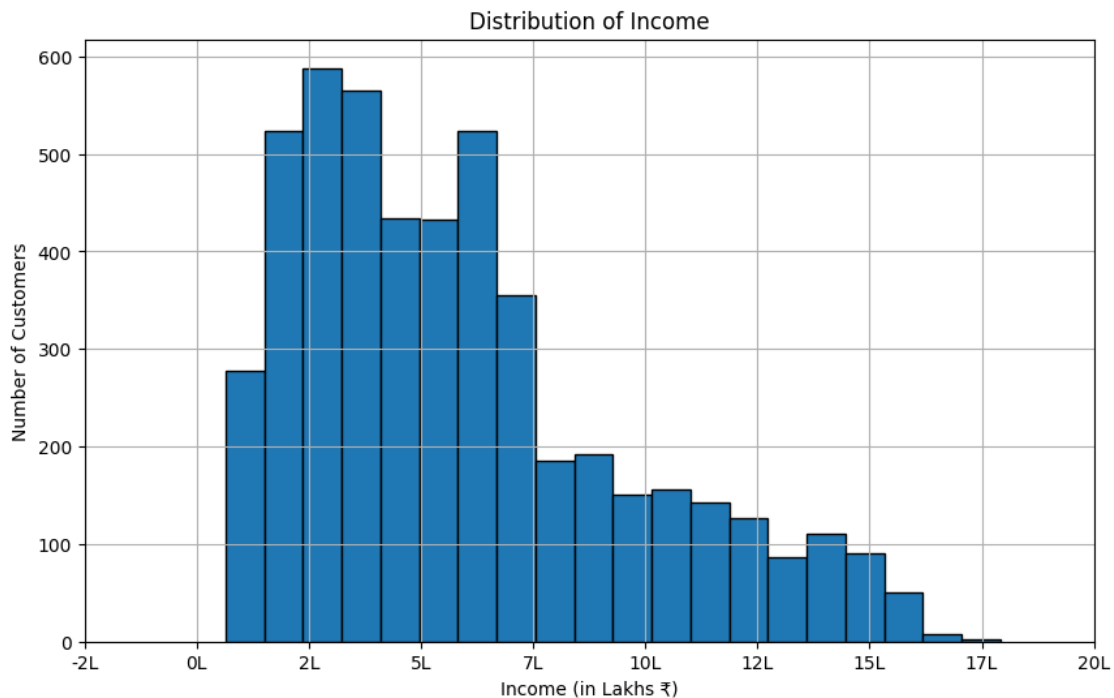
Income: Skewed to the right, with a few individuals having very high incomes.

Mortgage: Skewed to the right, with a few individuals having very high mortgages.

Family members and other variables: No clear linear relationships are observed, indicating that family size might not be strongly correlated with other variables in this dataset.

1.9.3 Distribution of Income

```
[20]: plt.figure(figsize=(10, 6))
plt.hist(df['Income'], bins=20, edgecolor='k')
plt.title('Distribution of Income')
plt.xlabel('Income (in Lakhs )')
plt.ylabel('Number of Customers')
ax = plt.gca()
ax.set_xticks(ax.get_xticks())
ax.set_xticklabels([f'{int(x/100000)}L' for x in ax.get_xticks()])
plt.grid(True)
plt.show()
```



We can see from this histogram that most of the customers are based around 2-6 Lacs per annum range. This indicates that the customer base primarily consists of lower to middle income groups.

Lets look at more detailed analysis of the Income distribution

```
[21]: #Defining income brackets to segregate into different income groups
df_1 = df
bins = [0, 200000, 500000, 1000000, df_1['Income'].max()]
labels = ['Low Income', 'Middle Income', 'Upper Middle Income', 'High Income']
df_1['Income Category'] = pd.cut(df_1['Income'], bins=bins, labels=labels,
    include_lowest=True)
# Number of customers in each category
income_distribution = df_1['Income Category'].value_counts()
```

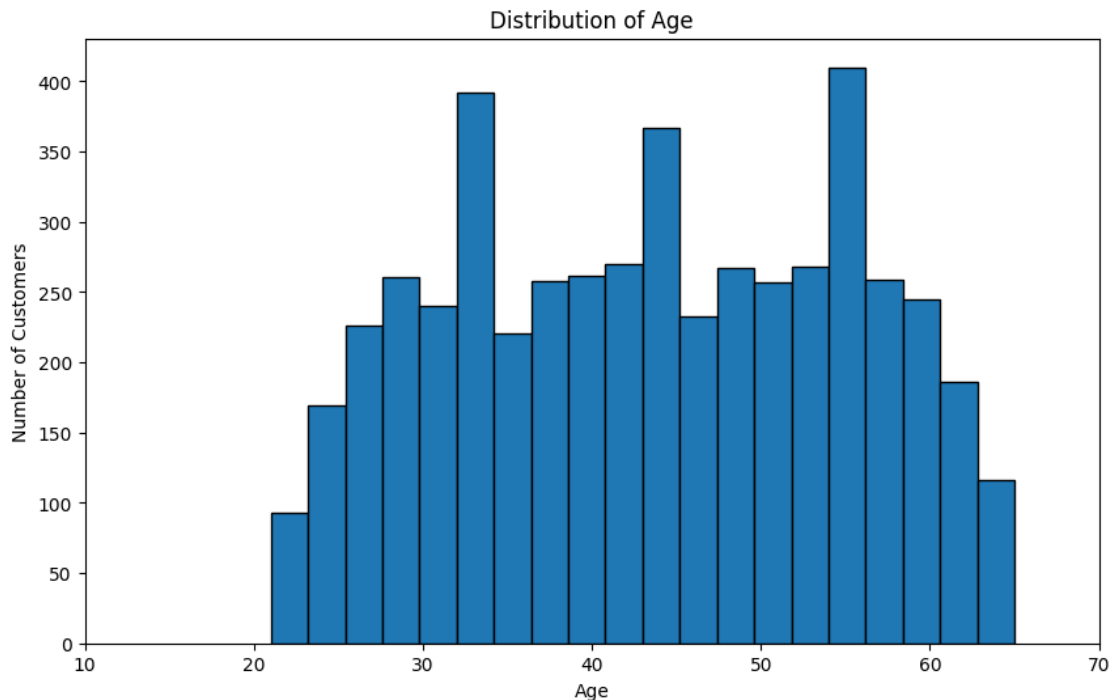
```
print(income_distribution)
```

```
Income Category
Upper Middle Income    1785
Middle Income          1772
High Income            771
Low Income             672
Name: count, dtype: int64
```

This allows us to see the exact number of customers in the Income section of the dataset. This confirms that majority of the customer base exists in the middle income range.

1.9.4 Distribution of Age

```
[22]: plt.figure(figsize=(10, 6))
plt.hist(df['age'], bins=20, edgecolor='k')
plt.title('Distribution of Age')
plt.xlabel('Age')
plt.ylabel('Number of Customers')
ax = plt.gca()
ax.set_xticks(ax.get_xticks())
plt.show()
```



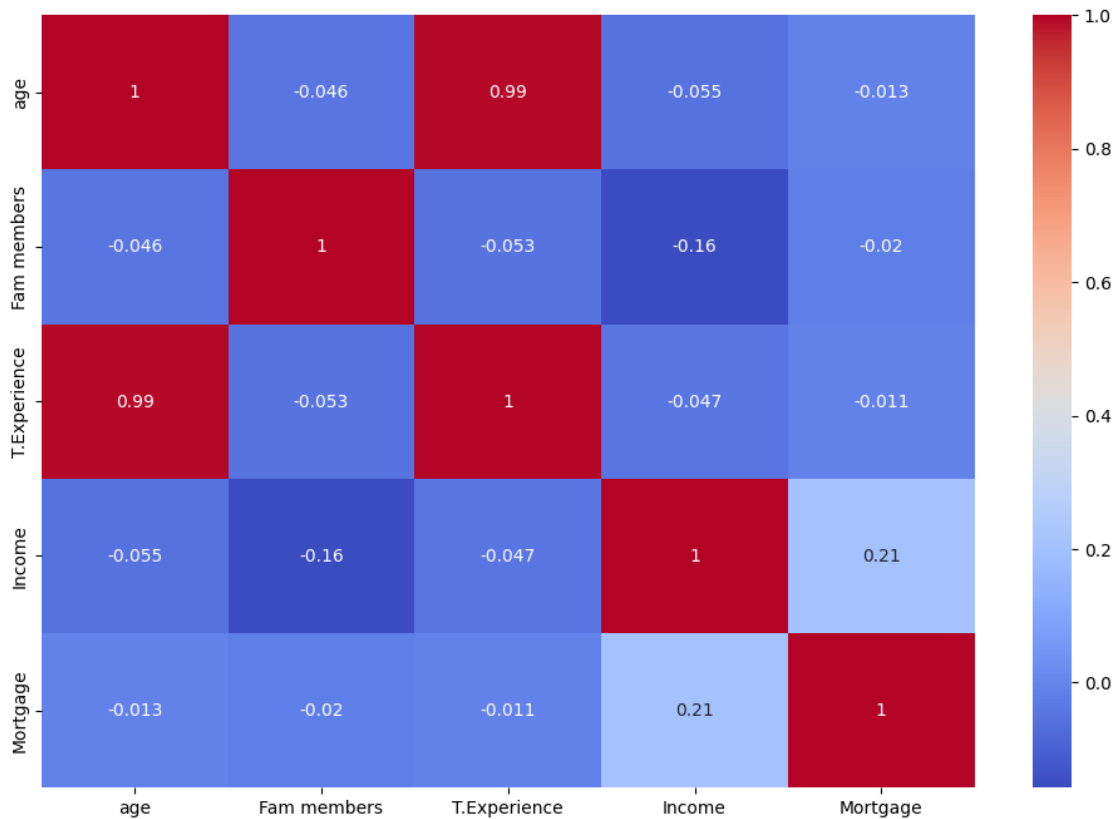
From this histogram, we can conclude that:

Majority of customer age is around 30 to 55 years of age.

There are very few customers who are above 60 years of age or below 25 years of age.

1.9.5 Finding Relation between all the different categories

```
[23]: plt.figure(figsize=(12, 8))
sns.heatmap(df[['age', 'Fam members', 'T.Experience', 'Income', 'Mortgage']].
            corr(), annot=True, cmap='coolwarm')
plt.show()
```



1.10 In short,

Bank mostly serves nuclear families(1-4 members)

Ages range from 21 to 65 years and peak around 30 to 55 years of age

Income levels of bank's customers are diverse

Individuals with higher income tend to take out larger loans

Individuals with larger fixed deposits might be less likely to take out loans

Customers who use online banking are more likely to apply for loans online

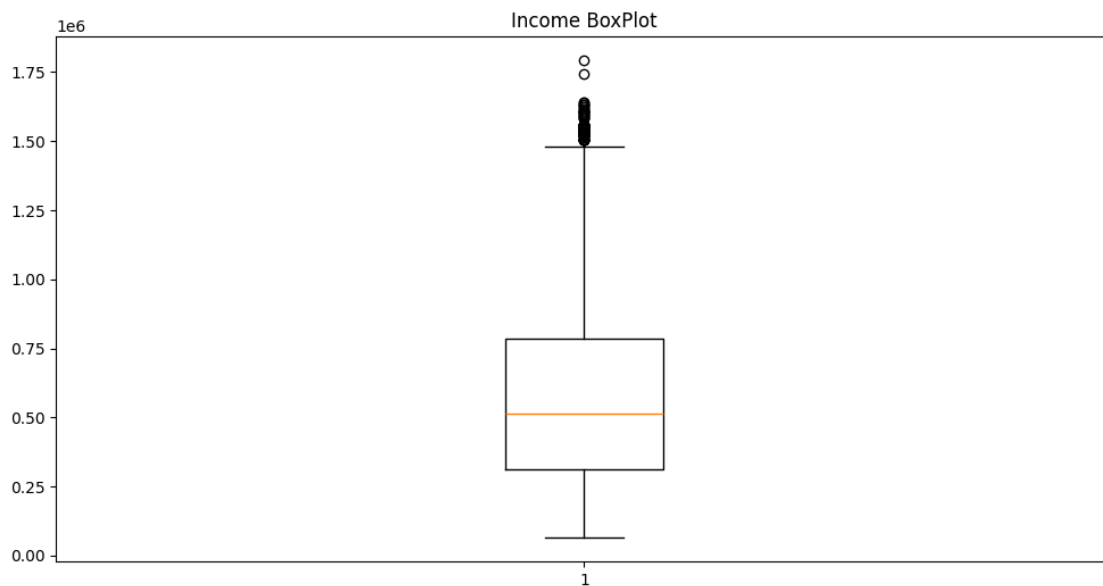
Individuals with lower education levels are more likely to take out loans

Bank serves customer mostly based around 2 to 6 lacs per annum range while catering to high and low ends too.

We noticed that major influences on the Loan variable were Income and Mortgage
Let us explore their Box Plots to make sure there are no outliers/skewness

```
[24]: plt.figure(figsize=(12,6))
plt.title("Income BoxPlot")
plt.boxplot(df['Income'])
```

```
[24]: {'whiskers': [<matplotlib.lines.Line2D at 0x243ab6c66f0>,
<matplotlib.lines.Line2D at 0x243ab71fc80>],
'caps': [<matplotlib.lines.Line2D at 0x243ab71fe90>,
<matplotlib.lines.Line2D at 0x243ab75c1a0>],
'boxes': [<matplotlib.lines.Line2D at 0x243ab71f860>],
'medians': [<matplotlib.lines.Line2D at 0x243ab75c410>],
'fliers': [<matplotlib.lines.Line2D at 0x243ab75c6e0>],
'means': []}
```



We notice that Income has some outliers and is skewed slightly

We apply Log Transformation to handle the outliers

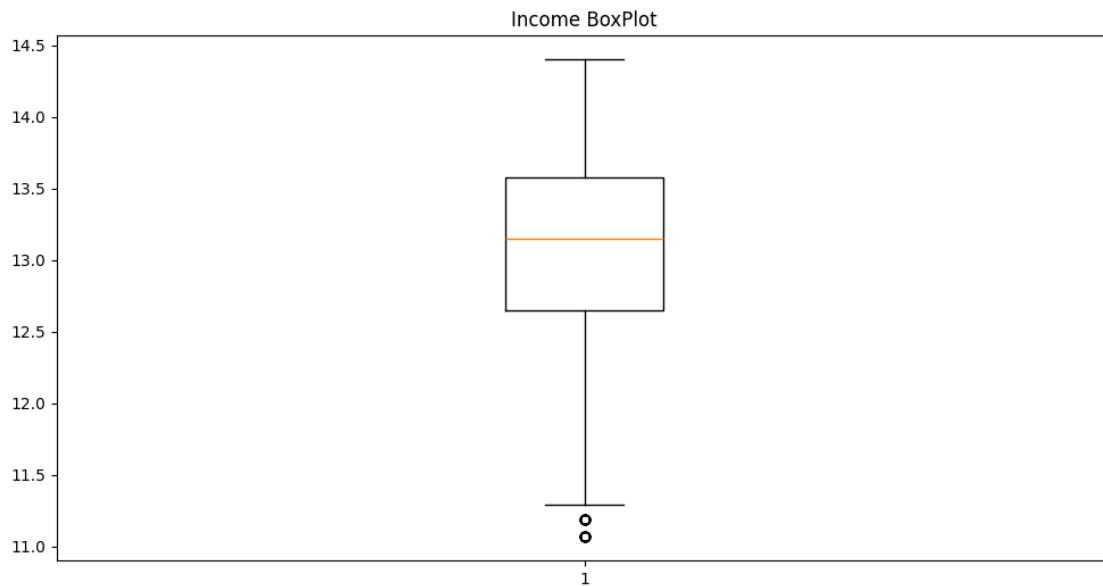
```
[25]: df['Income'] = df['Income'].apply(lambda x: np.log1p(x))
```

Re-checking the BoxPlot of Income

```
[26]: plt.figure(figsize=(12,6))
plt.title("Income BoxPlot")
```

```
plt.boxplot(df['Income'])
```

```
[26]: {'whiskers': [<matplotlib.lines.Line2D at 0x243ab786ba0>,  
  <matplotlib.lines.Line2D at 0x243ab786e70>],  
  'caps': [<matplotlib.lines.Line2D at 0x243ab787140>,  
  <matplotlib.lines.Line2D at 0x243ab787410>],  
  'boxes': [<matplotlib.lines.Line2D at 0x243ab786900>],  
  'medians': [<matplotlib.lines.Line2D at 0x243ab787650>],  
  'fliers': [<matplotlib.lines.Line2D at 0x243ab7878f0>],  
  'means': []}
```



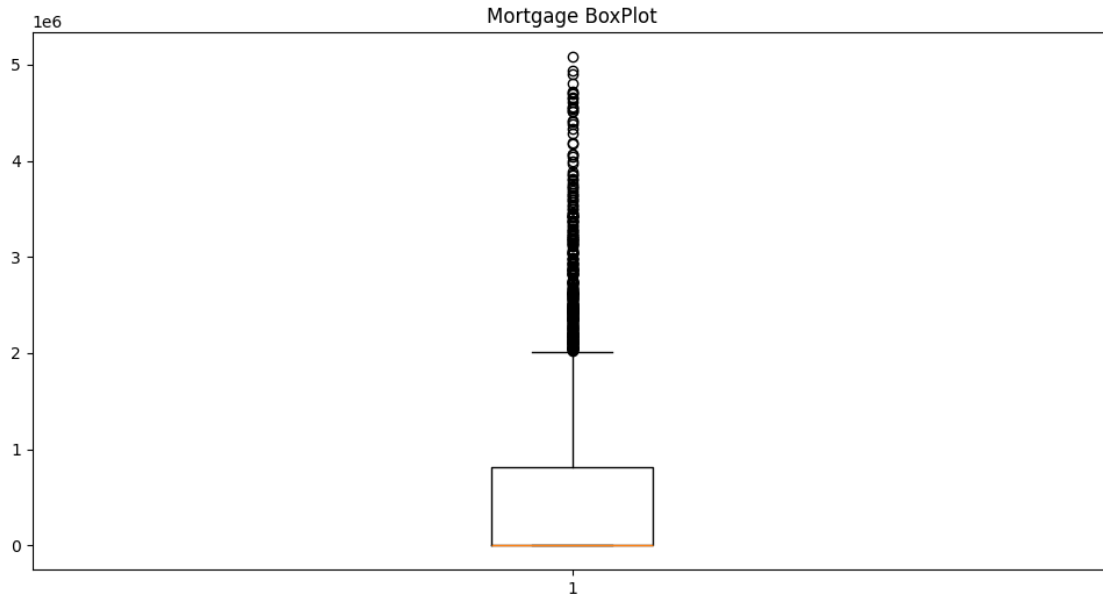
As we can see, the outliers have been reduced and the plot is no longer skewed.

Coming on to Mortgage feature

Let us plot the BoxPlot of Mortgage this time

```
[27]: plt.figure(figsize=(12,6))  
plt.title("Mortgage BoxPlot")  
plt.boxplot(df['Mortgage'])
```

```
[27]: {'whiskers': [<matplotlib.lines.Line2D at 0x243ab811d60>,  
  <matplotlib.lines.Line2D at 0x243ab811fa0>],  
  'caps': [<matplotlib.lines.Line2D at 0x243ab812210>,  
  <matplotlib.lines.Line2D at 0x243ab812510>],  
  'boxes': [<matplotlib.lines.Line2D at 0x243ab811c10>],  
  'medians': [<matplotlib.lines.Line2D at 0x243ab812780>],  
  'fliers': [<matplotlib.lines.Line2D at 0x243ab812a50>],  
  'means': []}
```



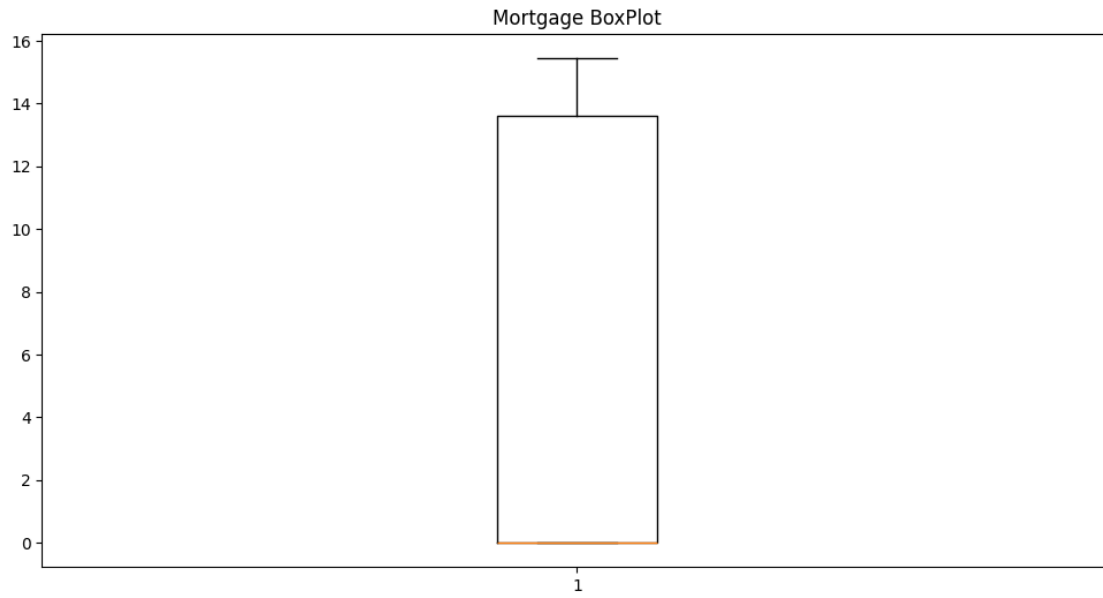
This plot means the presence of numerous outliers

Presence of outliers impacts linear models severely. We apply the same steps as above

```
[28]: df['Mortgage'] = df['Mortgage'].apply(lambda x: np.log1p(x))
```

```
[29]: plt.figure(figsize=(12,6))
plt.title("Mortgage BoxPlot")
plt.boxplot(df['Mortgage'])
```

```
[29]: {'whiskers': [<matplotlib.lines.Line2D at 0x243ab7c6120>,
<matplotlib.lines.Line2D at 0x243ac763aa0>],
'caps': [<matplotlib.lines.Line2D at 0x243ac763c20>,
<matplotlib.lines.Line2D at 0x243ac763ef0>],
'boxes': [<matplotlib.lines.Line2D at 0x243ac763ad0>],
'medians': [<matplotlib.lines.Line2D at 0x243ac7a0170>],
'fliers': [<matplotlib.lines.Line2D at 0x243ac7a0440>],
'means': []}
```



All the outliers have been handled.

1.11 Preprocessing the dataset

Now that we have finished EDA and gained valuable insights, now its time to begin preprocessing
Splitting the target variable “Loan” from the main dataset

```
[30]: target = df['Loan']
      features = df.drop(columns=['Income Category', 'Loan', 'ID', 'Pin-code'])
```

Verifying the split

```
[31]: target.info()

<class 'pandas.core.series.Series'>
RangeIndex: 5000 entries, 0 to 4999
Series name: Loan
Non-Null Count  Dtype
-----
5000 non-null   int32
dtypes: int32(1)
memory usage: 19.7 KB
```

```
[32]: features.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 11 columns):
#   Column              Non-Null Count  Dtype
---

```

```

---      -----      -----      -----
0   age           5000 non-null   int64
1   Fam members   5000 non-null   float64
2   T.Experience  5000 non-null   float64
3   Income        5000 non-null   float64
4   Mortgage      5000 non-null   float64
5   Fixed Deposit  5000 non-null   int32
6   Demat         5000 non-null   int32
7   Net Banking   5000 non-null   int32
8   Education_Graduate  5000 non-null   int64
9   Education_Post Graduate  5000 non-null   int64
10  Education_Under Graduate  5000 non-null   int64
dtypes: float64(4), int32(3), int64(4)
memory usage: 371.2 KB

```

```
[33]: features.head()
```

```

[33]:   age  Fam members  T.Experience   Income  Mortgage  Fixed Deposit  Demat  \
0    23           4.0           1.0  12.879020         0.0             0         1
1    43           3.0          19.0  12.513561         0.0             0         1
2    37           1.0          15.0  11.385103         0.0             0         0
3    33           1.0           9.0  13.592368         0.0             0         0
4    33           4.0           8.0  12.793862         0.0             0         0

      Net Banking  Education_Graduate  Education_Post Graduate  \
0              0              0              0
1              0              0              0
2              0              0              0
3              0              1              0
4              0              1              0

      Education_Under Graduate
0              1
1              1
2              1
3              0
4              0

```

Splitting the dataset into Train-Test Pairs

```
[34]: X_train, X_test, y_train, y_test = train_test_split(features, target,
↳ test_size=0.2, random_state=30)
```

Verifying the split

```
[35]: X_train.info()
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 4000 entries, 1237 to 4517

```

Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	age	4000 non-null	int64
1	Fam members	4000 non-null	float64
2	T.Experience	4000 non-null	float64
3	Income	4000 non-null	float64
4	Mortgage	4000 non-null	float64
5	Fixed Deposit	4000 non-null	int32
6	Demat	4000 non-null	int32
7	Net Banking	4000 non-null	int32
8	Education_Graduate	4000 non-null	int64
9	Education_Post Graduate	4000 non-null	int64
10	Education_Under Graduate	4000 non-null	int64

dtypes: float64(4), int32(3), int64(4)
memory usage: 328.1 KB

[36]: X_test.info()

```
<class 'pandas.core.frame.DataFrame'>  
Index: 1000 entries, 2192 to 3550  
Data columns (total 11 columns):  
#    Column                Non-Null Count  Dtype  
---  -  
0    age                    1000 non-null   int64  
1    Fam members            1000 non-null   float64  
2    T.Experience            1000 non-null   float64  
3    Income                  1000 non-null   float64  
4    Mortgage                1000 non-null   float64  
5    Fixed Deposit           1000 non-null   int32  
6    Demat                   1000 non-null   int32  
7    Net Banking             1000 non-null   int32  
8    Education_Graduate      1000 non-null   int64  
9    Education_Post Graduate 1000 non-null   int64  
10   Education_Under Graduate 1000 non-null   int64  
dtypes: float64(4), int32(3), int64(4)  
memory usage: 82.0 KB
```

[37]: y_train.info()

```
<class 'pandas.core.series.Series'>  
Index: 4000 entries, 1237 to 4517  
Series name: Loan  
Non-Null Count  Dtype  
-----  
4000 non-null   int32  
dtypes: int32(1)  
memory usage: 46.9 KB
```

```
[38]: y_test.info()
```

```
<class 'pandas.core.series.Series'>
Index: 1000 entries, 2192 to 3550
Series name: Loan
Non-Null Count  Dtype
-----
1000 non-null   int32
dtypes: int32(1)
memory usage: 11.7 KB
```

The Train-Test pairs both add up to the original rows, i.e. 5000.

The splits were successful.

1.12 Building the model

First, lets prepare a baseline model

1.13 Logistic Regression

```
[39]: # We will now initialize and train the model
model = LogisticRegression(max_iter=500)
```

```
[40]: #Fitting the model on split data
model.fit(X_train, y_train)
```

```
[40]: LogisticRegression(max_iter=500)
```

```
[41]: # Predictions and evaluation
y_pred = model.predict(X_test)
print(f'Accuracy: {accuracy_score(y_test, y_pred)}')
```

Accuracy: 0.97

Logistic Regression gives a 97% accuracy!

Although the model is already so accurate with default parameters, we can employ advanced features such as grid search and polynomial features to help model capture complex, non-linear relationship between the features.

Now lets try different models to find the best accuracy across all different models

Decision Tree

```
[42]: tree = DecisionTreeClassifier(random_state=30)
param_grid_tree = {
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': [None, 'sqrt', 'log2'],
    'splitter': ['best', 'random'],
```

```

        'criterion': ['gini', 'entropy']
    }
    grid_search_tree = GridSearchCV(estimator=tree, param_grid=param_grid_tree,
    ↪cv=5, scoring='accuracy')
    grid_search_tree.fit(X_train, y_train)
    print("Best Parameters:", grid_search_tree.best_params_)
    print("Best Accuracy:", grid_search_tree.best_score_)

```

Best Parameters: {'criterion': 'entropy', 'max_depth': 10, 'max_features': None, 'min_samples_leaf': 2, 'min_samples_split': 5, 'splitter': 'best'}

Best Accuracy: 0.9800000000000001

We found the best parameters for Decision Tree

1.13.1 Random Forest

Next model we will fit is random forest ensemble

```

[43]: rf = RandomForestClassifier(random_state=30)
    param_grid_rf = {
        'n_estimators': [100, 200],
        'max_depth': [None, 10, 20],
        'min_samples_split': [2, 5],
        'min_samples_leaf': [1, 2],
        'max_features': ['sqrt', 'log2'],
    }

    grid_search_rf = GridSearchCV(estimator=rf, param_grid=param_grid_rf, cv=5,
    ↪scoring='accuracy')
    grid_search_rf.fit(X_train, y_train)
    print("Best Parameters:", grid_search_rf.best_params_)
    print("Best Accuracy:", grid_search_rf.best_score_)

```

Best Parameters: {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}

Best Accuracy: 0.9817499999999999

1.13.2 Scaling data

```

[44]: scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

```

1.14 Voting Classifier

Using the best parameters from all the different models used till now to make the model more accurate

```
[45]: # Choosing base models
logreg = LogisticRegression(C=1, max_iter=200, penalty='l1', solver='liblinear')
tree = DecisionTreeClassifier(criterion='entropy', max_depth=10,
    ↪max_features=None, min_samples_leaf=1, min_samples_split=10, splitter='best')
forest = RandomForestClassifier(max_features='sqrt', n_estimators=200)
#nn = MLPClassifier(activation='relu', alpha=0.0001, hidden_layer_sizes=(100,
    ↪), max_iter=2000)
voting_clf = VotingClassifier(estimators=[
    ('logreg', logreg),
    ('tree', tree),
    ('forest', forest)
], voting='soft')
voting_clf.fit(X_train_scaled, y_train)
y_pred = voting_clf.predict(X_test_scaled)
print(f'Voting Classifier Accuracy: {accuracy_score(y_test, y_pred)}')
```

Voting Classifier Accuracy: 0.982

1.15 Applying polynomial features

to Logistic Regression to capture complex relationships

```
[50]: polynomial_pipeline = Pipeline([
    ('poly', PolynomialFeatures(degree=2, include_bias=False)),
    ('scaler', StandardScaler()),
    ('logistic', LogisticRegression(solver='liblinear'))
])

polynomial_pipeline.fit(X_train, y_train)
y_pred = polynomial_pipeline.predict(X_test)

print(f'Accuracy with Polynomial Features: {accuracy_score(y_test, y_pred)}')
```

Accuracy with Polynomial Features: 0.986

This is the highest accuracy we have achieved out of all the models.

1.16 Conclusion

1. Income, Fixed Deposits and Mortgage are the most important variables as they show high correlation with Loan.

1.17 Income > Fixed Deposit > Mortgage

2. Logistic Regression algorithm with fine tuning showed 98.6% accuracy

2 Thank You

Submitted for the XXXXXXXXXX hackathon by Anshuman Singh